

EchoSphere v2.2: Hardware Connection & Speaker Subsystem — Developer Implementation Handoff Guide

Technical Task Specification & Code Implementation Handoff Manual for Hardware Engineer

1. EXECUTIVE TASK OVERVIEW

The database models (speaker_queue.py), Flutter UI layout (speaker_queue_page.dart), and main API architecture are built. Your task is to complete the **Hardware Connection Tier** by implementing the backend endpoints, TTS audio service, MQTT broker driver, physical ESP32 firmware, and dynamic Flutter connection.

2. BACKEND HARDWARE API ENDPOINTS TO BUILD

HTTP Method	Route	Authorization Role	Description & Logic
GET	`/api/v1/hardware/speakers`	HoD, Admin, DevAdmin	Query PostgreSQL for all registered speaker nodes & heartbeat status.
POST	`/api/v1/hardware/speakers/register`	College Admin, DevAdmin	Register new speaker MAC address, IP, and bind to `department_id`.
POST	`/api/v1/hardware/speakers/{id}/broadcast`	HoD, Admin, Principal	Trigger immediate audio playback of approved notice on speaker node.
POST	`/api/v1/hardware/speakers/override`	Admin, Principal	Issue immediate EMERGENCY_OVERRIDE command across all department speakers.

3. TEXT-TO-SPEECH (TTS) VOICE SYNTHESIS SERVICE

```
# Create backend/app/services/tts_service.py
import os, gtts
AUDIO_DIR = 'backend/app/static/audio_streams'
async def generate_announcement_audio(announcement_id: int, text: str) -> str:
    os.makedirs(AUDIO_DIR, exist_ok=True)
    file_path = os.path.join(AUDIO_DIR, f'announcement_{announcement_id}.mp3')
    tts = gtts.gTTS(text=text, lang='en', slow=False)
    tts.save(file_path)
    return file_path
```

4. MQTT HARDWARE COMMUNICATION SERVICE

```
# Create backend/app/services/hardware_speaker_service.py
import json, paho.mqtt.client as mqtt
def dispatch_speaker_audio_command(dept_code: str, announcement_id: int, audio_url: str, is_emergency: bool = False):
    client = mqtt.Client()
    client.connect('localhost', 1883, 60)
    topic = f'echosphere/dept/{dept_code}/speakers/command'
    payload = { 'command': 'PLAY_EMERGENCY' if is_emergency else 'PLAY_ANNOUNCEMENT', 'announcement_id': announcement_id, 'audio_url': audio_url, 'volume': 100 }
    if is_emergency else 85 }
    client.publish(topic, json.dumps(payload), qos=1)
    client.disconnect()
```

5. MICROCONTROLLER FIRMWARE SCRIPT (ESP32 / RASPBERRY PI)

```
# Hardware Client Python/MicroPython Script (speaker_node_client.py)
import time, json, urllib.request, paho.mqtt.client as mqtt, pygame
DEPT_CODE = 'CSE'
pygame.mixer.init()
def on_message(client, userdata, msg):
    data = json.loads(msg.payload.decode())
    urllib.request.urlretrieve(data['audio_url'], '/tmp/announcement.mp3')
    pygame.mixer.music.load('/tmp/announcement.mp3')
    pygame.mixer.music.play()
client = mqtt.Client(client_id=f'Speaker_{DEPT_CODE}')
client.on_message = on_message
client.connect('192.168.1.100', 1883, 60)
client.subscribe(f'echosphere/dept/{DEPT_CODE}/speakers/command')
client.loop_start()
```